



CASE FILE MANAGEMENT & ANALYTICS PORTAL

Team ID: 11 | Section: 10 | Members: P.Ruhitha -2520080053, M.Ananya-2520080069, S.Himasree-2520090181 | Guide: [GUIDE NAME]

Database Systems Engineering and Distributed Backend Development (25CS1302E) | KLJH Bachupally Campus, CSE | PBL 2026-27

Abstract

- Role-based portal for lawyers and clients to manage cases, hearings, documents and judgments. Built with React, FastAPI, Spring Boot microservices and MySQL.
- One database and live analytics replace scattered files.

Problem Statement & Objectives

- **Problem:** Case data is scattered across files and chats, so hearings are missed.
- **Objective 1:** One consistent database for all case records.
- **Objective 2:** Independent User, Case and Record services over REST.
- **Objective 3:** Live analytics from the current database.
- **Scope:** Lawyer and client roles, filing to judgment.

Existing vs Proposed System

Feature	Existing System	Proposed System
Case tracking	Paper, spreadsheets	Single MySQL database
Hearings	Manual date notes	Scheduled per judge
Client visibility	Phone calls to office	Client dashboard
Analytics	None / manual counts	Live dashboard from SQL
Gap	No cross-record checks	REST validation between services

Feasibility Study

- **Technical:** All skills and tools are available; stack is React, FastAPI, Spring Boot, MySQL.
- **Economic:** Fully open-source stack and a local deployment, so no licence cost.
- **Operational:** Simple role-based dashboards; clients need no training.
- **Schedule:** Built and tested in phases within the trimester.

Results & Testing

- 20 cases, 17 users, 7 hearings, 5 documents and 4 judgments verified.
- Health checks, CRUD, REST validation and UNIQUE-constraint tests passed.
- Frontend build compiled (2441 modules).

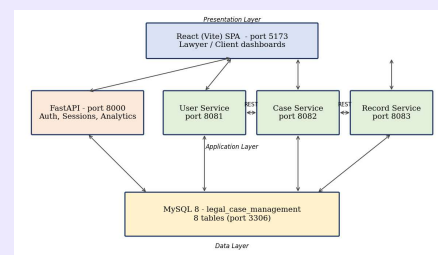
Conclusion

- Delivered a working role-based portal with hashed (Argon2id) passwords.
- Meets all objectives: validated relationships, microservices and live analytics.

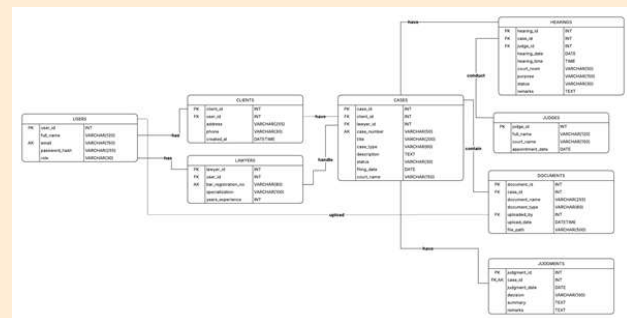
Tools & Technologies

- **Front-End:** React (Vite), Context API, Recharts
- **Back-End:** FastAPI (auth, analytics) + 3 Spring Boot microservices over REST
- **Database:** MySQL 8 – 8 tables, PK / FK / UNIQUE constraints
- **Tools:** Git & GitHub, Postman / Swagger, VS Code, MySQL Workbench

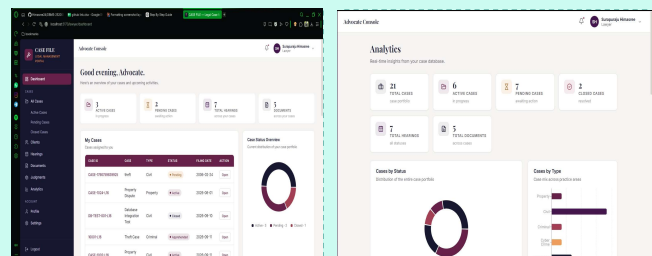
System Architecture



ER Diagram & Database Design



Implementation Screenshots



Future Scope

- Real document upload / download service.
- Hearing reminders by e-mail / SMS and a calendar view.
- Docker, API gateway and persistent sessions.

